

PRACTICAL 3 DELIVERABLE

UML Design & Software Architecture Report

Comprehensive System Modeling, Architectural Diagrams, SRS, and Engineering Artifacts for the Tevar E-Commerce Platform.

PROJECT	SUBJECT	TOPOLOGY	STATUS
Tevar Fashion	Software Engineering Lab	Next.js + Express + Neon DB	Production Grade

1. Problem Analysis & Requirement Gathering (SRS)

1.1 Problem Statement: Modern fashion e-commerce demands high-availability browsing, instant inventory reservation, secure multi-channel payments, and granular administrative control. Monolithic architectures suffer from performance bottlenecks during flash sales, tight coupling between catalog and order management, and fragile state synchronization.

1.2 Core System Objectives:

- **Micro-Modular Decoupled Architecture:** Decouple storefront UI (Next.js/Vercel) from core business logic (Express REST API on Render).
- **ACID Transaction Integrity:** Guarantee atomic state changes across order creation, inventory stock deduction, and Razorpay webhook capture.
- **Low-Latency Catalog & Media CDN:** Zero-egress Cloudflare R2 object storage with Redis-backed rate limiting and session caching.
- **Role-Based Access Control (RBAC):** Granular privileges for `CUSTOMER` , `ADMIN` , and `SUPER_ADMIN` roles.

1.3 Functional Requirements (FR)

ID	Requirement Name	Description	Actors
FR-01	User Auth & Profile	JWT dual-token auth (Access 15m + Refresh 7d), Bcrypt hashing, address book management.	Customer, Admin
FR-02	Catalog Browsing & Search	Multi-facet product filtering (category, size, color, price) and full-text keyword search.	Customer, Guest
FR-03	Cart & Wishlist	Real-time cart sync, quantity modification, and persistent session storage.	Customer
FR-04	Order & Checkout Engine	Multi-item order calculation, automated tax/coupon evaluation, and stock reservation.	Customer, System
FR-05	Payment Gateway Integration	Razorpay order creation, client modal checkout, and HMAC-SHA256 signature verification.	Customer, Razorpay
FR-06	Inventory Control	Atomic decrement on payment success; auto-restocking on payment failure or timeout.	System, Admin
FR-07	Admin & Operations	Real-time sales dashboard, SKU management, order shipment fulfillment tracking.	Admin, Super Admin

2. UML Diagrams & Structural Modeling

2.1 Use Case Diagram

Defines functional requirements and actor interaction boundaries for Storefront and Admin subsystems.

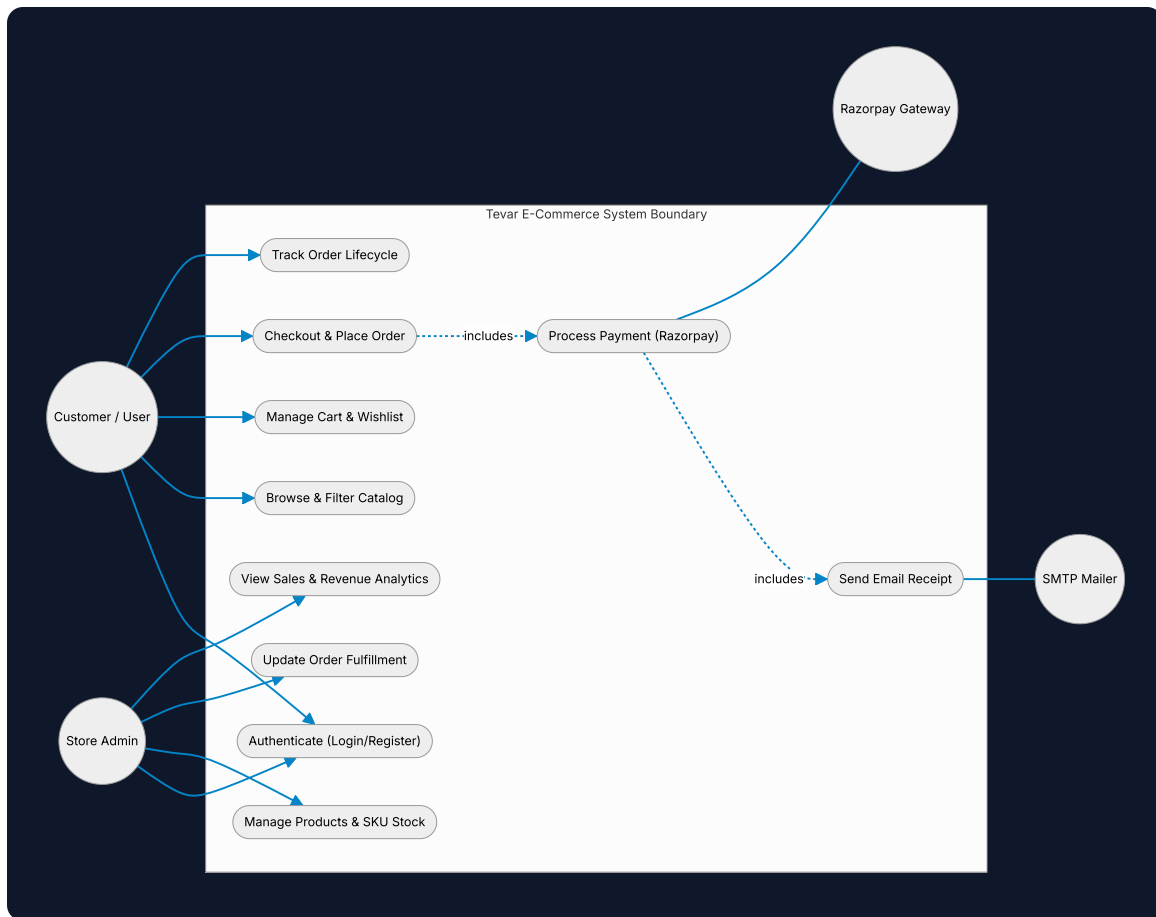


FIGURE 2.1 — USE CASE DIAGRAM FOR TEVAR E-COMMERCE SYSTEM

2.2 Class Diagram (Domain & Entity Model)

Represents the core data structures, properties, relationships, and service methods implemented in Prisma ORM.



FIGURE 2.2 — SYSTEM CLASS DIAGRAM (DOMAIN MODEL & ASSOCIATIONS)

2.3 Activity Diagram (Order Checkout & Payment Workflow)

Details the end-to-end decision logic, stock verification, and branching paths during customer checkout.



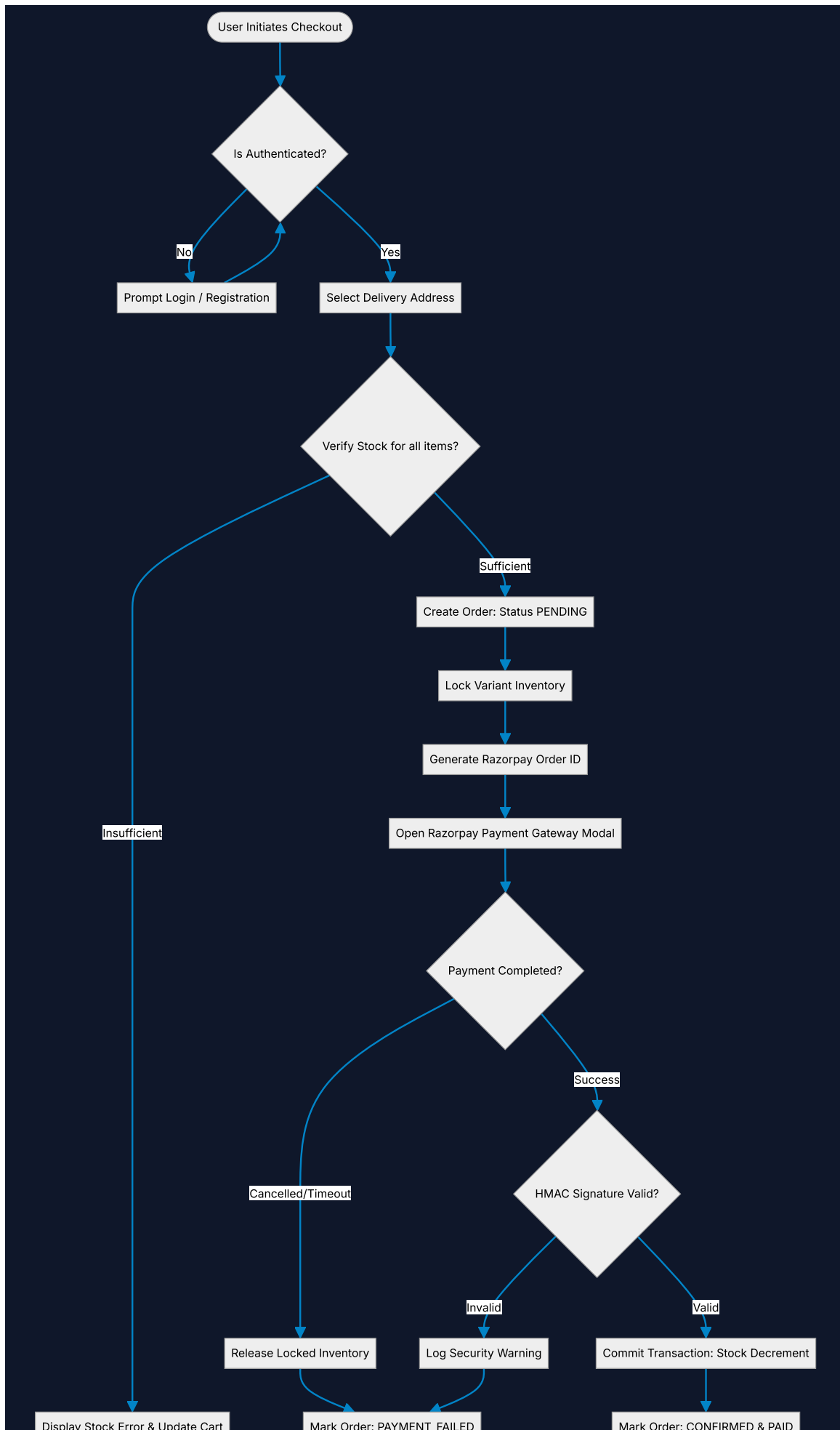




FIGURE 2.3 — ACTIVITY DIAGRAM FOR CHECKOUT & STOCK RESERVATION FLOW

2.4 Sequence Diagram (Checkout & Webhook Verification)

Models the chronological message flow across the client browser, Express backend, Razorpay gateway, and Neon PostgreSQL database.

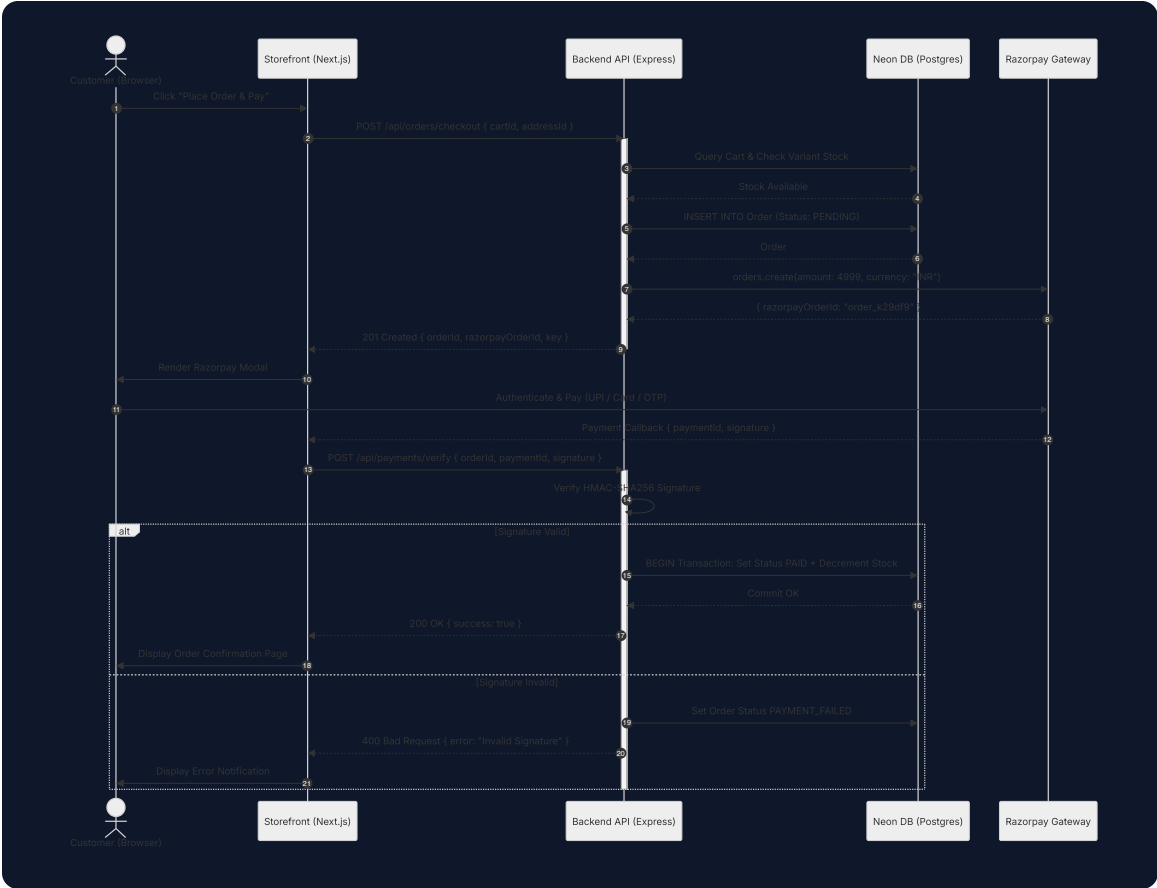


FIGURE 2.4 — SEQUENCE DIAGRAM FOR CHECKOUT & RAZORPAY SIGNATURE VERIFICATION

2.5 State Machine Diagram (Order Lifecycle)

Illustrates state transitions, triggers, and terminating states for an order from placement to delivery.



FIGURE 2.5 — STATE MACHINE DIAGRAM FOR ORDER LIFECYCLE

2.6 Component Diagram

Modular structural decomposition showing component interfaces, dependencies, and caching boundaries.

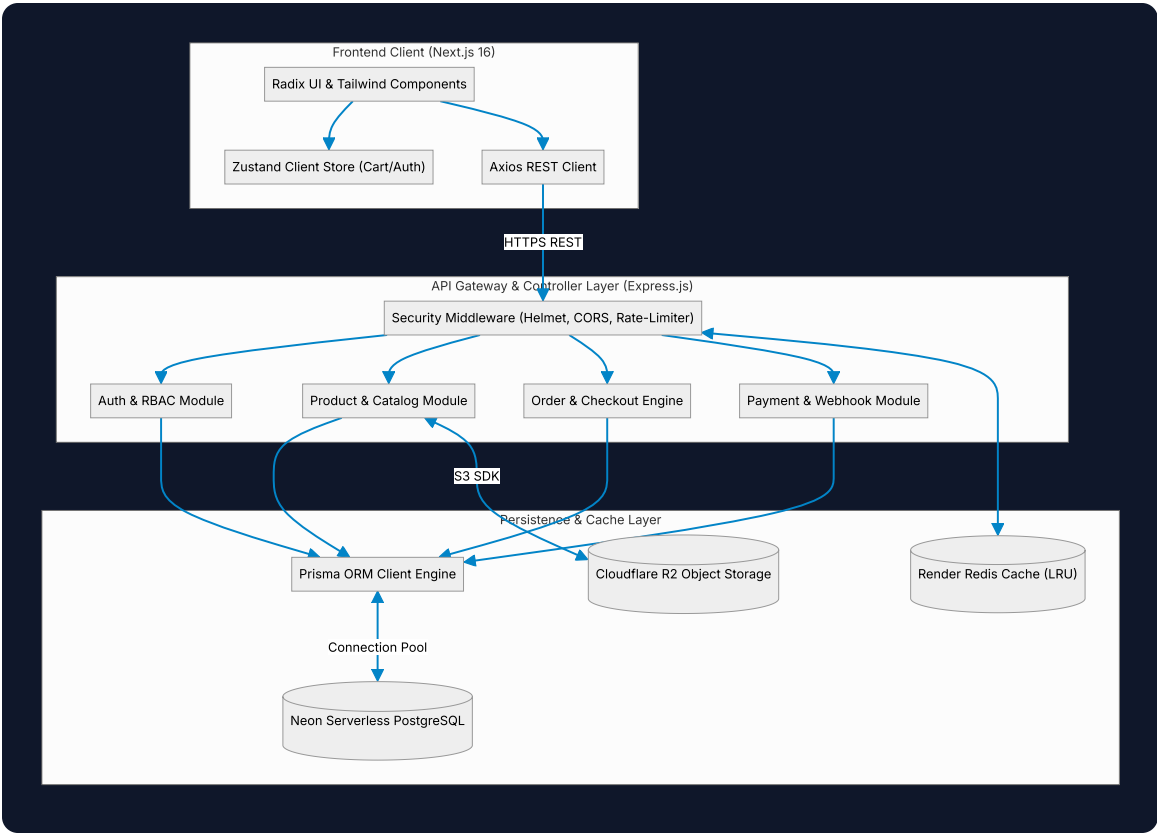


FIGURE 2.6 — SYSTEM COMPONENT DIAGRAM

2.7 Deployment Diagram

Physical deployment topology including global CDNs, cloud containers, databases, and third-party payment gateways.

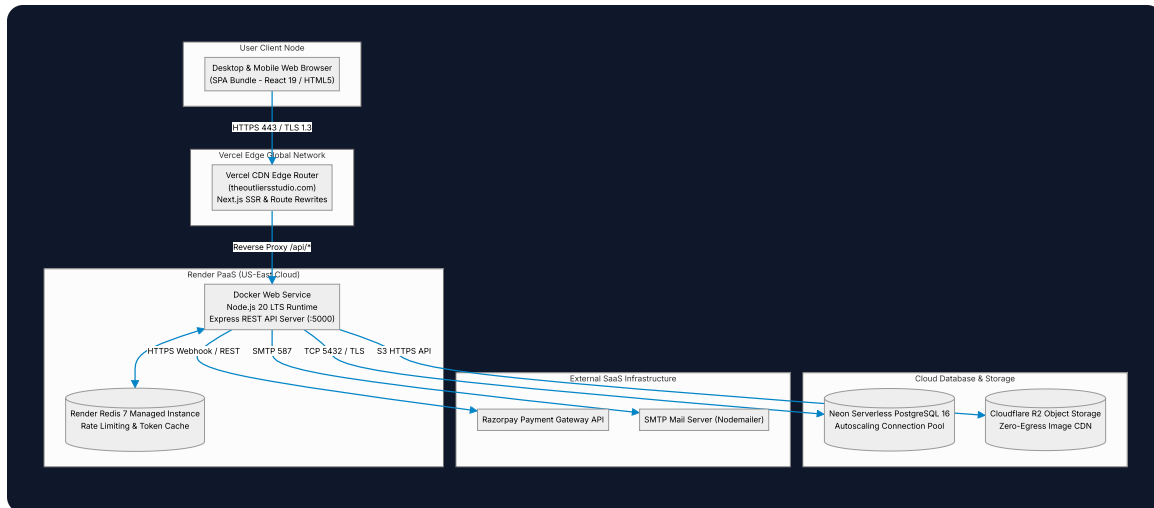


FIGURE 2.7 — DEPLOYMENT DIAGRAM (PHYSICAL INFRASTRUCTURE TOPOLOGY)

3. Software Engineering Tools & Technologies

Tool / Framework	Role in System	Engineering Rationale
Next.js 16 + React 19	Frontend Application	App router, server-side rendering for optimal SEO, and instant client-side navigation.
Node.js + Express.js	Backend REST API	Asynchronous non-blocking I/O architecture ideal for high-throughput e-commerce workloads.
Prisma ORM	Data Access Layer	Auto-generated type-safe queries, migration control, and transactional guarantees.
Neon DB (PostgreSQL 16)	Primary Database	Serverless PostgreSQL with instant compute branching, auto-scaling, and pooled connections.
Render Redis	In-Memory Cache	LRU rate-limiting engine (<code>rate-limit-redis</code>) preventing DDoS and brute-force attacks.
Cloudflare R2	Media Storage	S3-compatible bucket eliminating expensive data egress fees for high-resolution product media.
Razorpay API	Payment Gateway	PCI-DSS compliant payment processing supporting UPI, credit/debit cards, and net banking.
Git & GitHub	Version Control & CI	Branch protection rules, pull request audits, and automated lint/test workflows.

4. Project Planning & Agile Artifacts

4.1 Work Breakdown Structure (WBS)

```
1.0 Tevar E-Commerce System
├── 1.1 Requirements & System Architecture
│   ├── 1.1.1 Problem analysis & SRS documentation
│   └── 1.1.2 UML structural & behavioral modeling
├── 1.2 Frontend Storefront & Admin Portal
│   ├── 1.2.1 Tailwind CSS design tokens & responsive components
│   ├── 1.2.2 Product catalog, search & filtering UI
│   ├── 1.2.3 Zustand Cart, Wishlist & Checkout flows
│   └── 1.2.4 Admin dashboard (TanStack Table & Analytics charts)
├── 1.3 Backend REST API & Database
│   ├── 1.3.1 Prisma schema definition & Neon DB migrations
│   └── 1.3.2 JWT Authentication & RBAC middleware
```

- | | | 1.3.3 Products, Categories & Cloudflare R2 media upload
- | | | 1.3.4 Orders, Razorpay webhook & transactional stock decrement
- | | 1.4 Testing & Quality Assurance
 - | | | 1.4.1 Unit & integration tests (Supertest + Jest)
 - | | | 1.4.2 End-to-End checkout validation & payment mocking
 - | | | 1.4.3 Security audits (OWASP Top 10, rate limiting)
- | | 1.5 Cloud Deployment & CI/CD
 - | | | 1.5.1 Vercel Edge frontend setup & API route rewrites
 - | | | 1.5.2 Render Web Service & Redis instance provisioning

5. Testing Artifacts & Quality Assurance

5.1 Test Cases Specification

Test ID	Module	Test Description	Input Payload	Expected Result	Status
TC-01	Auth	Duplicate email registration	{ email: "user@brand.com" }	409 Conflict (User already exists)	PASS
TC-02	Catalog	Filter by non-existent size	GET /api/products?size=XXXXL	200 OK , products: []	PASS
TC-03	Inventory	Checkout item when stock is 0	variantId: 44, qty: 1 (Stock: 0)	400 Bad Request (Insufficient stock)	PASS
TC-04	Payment	Webhook with tampered signature	Altered HMAC payload	400 Bad Request (Invalid signature)	PASS
TC-05	Security	Rate limit threshold trigger	120 requests in 60s from 1 IP	429 Too Many Requests	PASS

5.2 Bug Report Sample

Bug ID: BUG-2026-084 | **Severity:** High | **Priority:** P1

Summary: Concurrency race condition allowed duplicate stock deduction during simultaneous checkout clicks.

Steps to Reproduce: Open two browser tabs with 1 unit remaining in stock. Click "Pay" simultaneously.

Root Cause: Lack of atomic locking between stock read and decrement operations in standard database queries.

Resolution: Encapsulated stock validation and quantity update inside a Prisma interactive transaction (`prisma.$transaction`).

6. Viva & Presentation Questions

Q1: Why separate the frontend on Vercel from the backend on Render instead of a Next.js fullstack monolith?

Answer: Decoupling allows the storefront to benefit from global Vercel Edge caching and static optimization, while the backend maintains long-lived TCP connection pools to PostgreSQL and Redis without serverless cold starts or strict timeout limits.

Q2: How does the system prevent payment fraud during checkout?

Answer: The server computes an HMAC-SHA256 hash using the private secret key, `razorpay_order_id`, and `razorpay_payment_id`, validating it against the received signature before committing the order to the database.